Unit-3: Regular Expression, Functions and File Handling

Defining and calling a function

A function can be defined using the keyword 'def' followed by function name. After writing a function name, we should write parentheses () in which parameters are written.

Syntax:

```
def functionname(parameter1, parameter2...):
    function statements
```

Example: #Program to add two numbers using function

```
def sum(a,b):
    c=a+b
    print('The sum of two digits: ',c)
#Calling the function
sum(10,15)
sum(2.5,5.2)
```

Returning results from a function

Example: Program to add two numbers using function, returning results from a function

```
def sum(a,b):
    c=a+b
    return(c)
#Calling the function
x=sum(10,15)
print('The sum is: ',x)
b=sum(2.5,5.2)
print('The sum is: ',b)
```

Example: Program to find whether the value is even or odd using function

```
def odd_even(num):
    if num%2==0:
        print('The', num, 'is Even')
    else:
        print('The', num, 'is Odd')
#Calling the function
odd_even(2)
odd_even(7)
```

#Program to check whether entered number is positive, negative or zero using function

```
def pos_neg_zero(num):
    if num>0:
        print('The number is positive')
    elif num==0:
        print('The number is 0')
    else:
        print('The number is negative')
#Calling the function
n=int(input('Enter a number to check: '))
pos_neg_zero(n)
```

Returning Multiple Values from a function

A function in Python can return multiple values. When function calculates multiple results and wants to return the results, we can use the return statement as:

```
return a,b,c
```

These values are returned by a function as a tuple.

Example: Performing basic arithmetic operations using function

```
def arith(a,b):
    add=a+b
    sub=a-b
    mul=a*b
    div=a/b
    return add,sub,mul,div
p,q,r,s=arith(10,5)
print('The result of addition is: ',p)
print('The result of subtraction is: ',q)
print('The result of subtraction is: ',r)
print('The result of subtraction is: ',s)
```

Pass by Object

Immutable objects in Python: integer, float, string, tuple

Mutable objects in Python: Lists

As the value assigned to variable a is of integer type, modifying the value will not be stored at a same memory address. As integer is immutable, modifying the value of 'a' will occupy another memory to store a new value.

Example:Program to pass an integer to a fucnction and modify it.

```
def modify(a):
    a=16
    print('The value of a inside the function is',a, id(a))
#Calling the function
a=7
modify(a)
print('The value of a outside the function is',a,id(a))
```

Example: Program to pass a list to a function and modify it.

```
def modify(lst):
    lst.append(15)
    print(lst,id(lst))
#Calling the function
lst=[10,11,12,13,14]
modify(lst)
print(lst,id(lst))
```

Formal and Actual Arguments

```
def sum(a,b):    →formal arguments
    ----
```

Calling the function

s=10, m=6 →actual arguments

The actual arguments in a function call are of 4 types:

-Positional arguments
-Keyword arguments
-Default arguments
-Variable length arguments

Positional arguments

The number of arguments and their positions in the function definition should match exactly with the number and position of the argument in the function call.

Example:

```
def combi(s1,s2):
s3=s1+s2
print(s3)
#Calling the function
combi('Good','Morning') #O/P: GoodMorning
combi('Morning','Good') #O/P: MorningGood
#combi('Good','Morning', 'all') #Error:combi() takes 2 positional arguments but 3 were given
```

Keyword arguments

Keyword arguments are arguments that identify the parameters by their names. At the time of calling this function, we have to pass two values and we can mention which value is for what.

Example:

```
def student(r_no,name):
print('Roll no. is :', r_no)
print('Name is :', name)
#Calling the function
student(r_no=1, name='abc')
student(r_no=2, name='cde')
```

Default arguments

If required we can set a default value to the parameter. If at the time of calling the function, the value is not passed to the argument than default value will be used. And if the value is passed to the argument than passed value will be used.

Example:

```
def student(r_no,name='Ram'):
print('Roll no. is :', r_no)
print('Name is :', name)
#Calling the function
student(r_no=1)
student(r_no=2, name='cde')
student(r_no=3)
```

Variable length arguments

In some situation it is possible that the programmer is unaware about the requirement of the parameters in the program. If in program, two parameters are declared, and while using it user feel to give more than two values than error will occur. In that case programmer may use variable length arguments. A variable length argument is an argument that can accept any number of values.

Format: `def name_of_func(farg, *args)`

Example:

```
def add(farg, *args):
#print('Formal argument= ', farg)
sum=0
```

```
for i in args:
        sum=sum+i
print('Sum of all numbers: ',(farg+sum))
#Calling the fucntion
add(10,20)
add(20,30,40)
```

Passing a Group of Elements to a Function

Some time it is required to receive group of elements like numbers or strings, we can accept them into a list and then pass the list to the function.

Example:

```
def show(lst):
for i in lst:
        print(i)
#Taking group of strings as an input
print('Enter strings seprated by space: ')
lst=[a for a in input().split( )]
show(lst)
```

<u>Anonymous Functions (Which is also known as Lambda)</u>

We already know while writing a function we need to give name to the function. We usually do it like:

def name_of_function(arguments..). But the anonymous function is not defined with 'def' it is defined with the keyword 'lambda'.

```
def square(a):
        return a*a
```

This could be written in anonymous function as : lambda a: a*a

Example: Find out the square of the entered number

```
square=lambda a:a*a
a=int(input('Enter the number: '))
print('The square of the entered number is: ', square(a))
```

Example: Find out the bigger number from entered two numbers

```
big=lambda a,b:a if a>b else b
a,b=[int(n) for n in input('Enter two numbers: ').split(',')]
print('The bigger number out of two numbers is:', big(a,b))
```

)

File Handling:

<u>Types of Files</u>

In Python, there are two types of files:

-Text files

-Binary files

Text files store the data in the form of characters. For example, if we store student name 'abc', it will be stored as 3 characters and the and his score is 87 than it is stored as 2 characters. Normally, text files are used to store characters or strings.

Binary files store entire data in the form of bytes, i.e. a group of 8 bits each. When the data is retrieved from the binary file, the programmer can retrieve the data as bytes. Binary files can be used to store text, images, audio, and video.

Opening and Closing file

We should use open() function to open a file. This function accepts 'filename' and 'openmode' in which to open the file.

file handler=open('file name', 'open mode')

File opening mode

w	To write data into a file. If any data is already present in the file, it would be deleted and the present data will be stored.
r	To read data from the file. The file pointer is positioned at the beginning of the file
a	To append data to the file. Appending at the end of existing data. File pointer is placed at the end of the file. If the file is not present, it will be created.
x	To open the file in exclusive creation mode. The file creation fails if the file already exists.
+	+ with w,r,a will allow both reading and writing

Closing a file:

A file which is opened should be closed using the close() method. If file is not closed, than data may get corrupt or we may lose data if not closed. Also if file not closed than it may not free the memory till the file is open. We can close file using f.close()

Example: Program to create a text file and store data

```
#Creating a file to store characters
#Open a file for writing data
f=open('myfile.txt','w')
#Accept text from the user
str=input('Enter the text: ')
#Writing string into file
f.write(str)
#Closing the file
f.close
```

Example: # Read the existing file

```
# Open the file for reading data
f=open('myfile.txt','r')
#Reading the file
str=f.read()
#Display them on the screen
print(str)
#Closing the file
f.close()
```

```
#Creating program to store a group of strings into a text file
#Open the file for writing data
f=open('myfile.txt','w')
#Enter strings from keyboard
print('Enter text you want to enter in your file: ')
while str!='$':
        str=input()  #Accept string into str
        if(str!='$'):
                f.write(str+"\n")
#Closing the file
 f.close()
```

```
#Append into an existing file
        f=open('myfile.txt','a+')
        #Enter strings from keyboard
        print('Enter text you want to enter in your file: ')
        while str!='$':
                str=input()  #Accept string into str
                if(str!='$'):
                        f.write(str+"\n")
        #Closing the file
        #f.close()
        f=open('myfile.txt','r')
        #Reading the file
        str=f.read()
        #Display them on the screen
        print(str)
        #Closing
        f.close()
```

Knowing whether a file Exists or Not

The operating system (os) module has a sub module by the name 'path' that contains a method isfile() method. This method can be used to know whether a file that we are opening really exists or not.
Syntax: os.path.isfile(fname) gives True if the file exists otherwise False.

```
#Checking whether the file is available or not if available, open it
        import os,sys
        #Enter the file name
        fname=input('Enter the file name to open: ')
        if os.path.isfile(fname):
                f=open('myfile.txt','r')
        else:
                print(fname+ 'does not exist')
                sys.exit()
        #Reading the file
        str=f.read()
        #Display them on the screen
        print(str)
        #Closing
        f.close()

#Counting number of lines, words, and characters if the file is available
        import os,sys
        #Enter the file name
        fname=input('Enter the file name to open: ')
        if os.path.isfile(fname):
                f=open('myfile.txt','r')
        else:
                print(fname+ 'does not exist')
                sys.exit()
        #Initialising the counters and setting it to zero
        cl=cw=cc=0
        #Reading lines from the file
        for line in f:
```

```
            words=line.split()
            cl=cl+1
            cw=cw+len(words)
            cc=cc+len(line)
    print('Number of lines in a file is: ',cl)
    print('Number of lines in a file is: ',cw)
    print('Number of lines in a file is: ',cc)
    #Closing
    f.close()
```

Working with Binary files

Binary files handle data in the form of bytes. It can store files like text, image, audio or video. To open a binary file for reading purpose, we can use 'rb'. Similarly to write bytes into binary file we can use 'wb'. To read bytes from a binary file, we can use the read() and to write bytes into a binary file, we can use the write() method.

Example: #To copy an image file into another file

```
    #Open the file in a Binary mode
    f1=open('Python Syllabus_Pg1.png','rb')
    f2=open('syllabus.png','wb')
    #Read bytes from f1 and write it to f2
    bytes=f1.read()
    f2.write(bytes)
    #Close the files
    f1.close()
    f2.close()
```

The with statement

The 'with' statement can be used while opening a file. The advantage of with statement is that it will take care of closing a file which is opened by it. The format of using 'with' is:

```
    With open('file.txt','openmode') as fileobject:
```

The seek() and tell() methods

We can use tell() method to know the position of the file pointer. It returns the current position of the file pointer from the beginning of the file. It is used in the form: n=f.tell()

If we want to move the file pointer to another position, we can uset the seek(). It is used in the form: f.seek(offset, fromwhere).

Here 'offset' represents how many bytes to move, and 'fromwhere' represents from which position to move. For example 'fromwhere' can be 0, 1, or 2. Here, 0 represents from the beginning of the file, 1 represents from the current position and 2 represents from the ending of the file. The default value is 0.

Example: #Taking data from the user and store in the form of binary file

```
    reclen=15  #It is a variable, Record length
    #Open the file in 'wb' mode
    with open ('stationary.bin','wb') as f:   #Use of 'with' will take care of closing the file
        n=int(input('How many items to add?: '))
        for i in range(n):
            stationary=input('Enter the name of stationary item: ')
            ln=len(stationary)   #To find the length of entered item
            #Add remaining spaces at the end to make it 15 chars
            stationary=stationary+(reclen-ln)*'  '
```

```
                    stationary=stationary.encode()   # encode() converts string to binary form
                    f.write(stationary)
```

Example: #To read the file by taking record number from the user

```
    reclen=15
    with open('stationary.bin','rb')as f:
        n=int(input('Enter record number to search: '))
        f.seek(reclen*(n-1))   #Moves the file pointer to the end of n-1 th record
        str=f.read(reclen)    #Get the nth record with 15 characters
        print(str.decode())   #Convert byte string into ordinary string
```

#Zipping the contents of files

```
    from zipfile import *
    #creating the zip file
    f=ZipFile('sample.zip','w',ZIP_DEFLATED)
    #Adding files to be zipped
    f.write('arr.py')
    f.write('ar2.py')
    #close the zip file
    f.close()
```

#Unzip

```
    from zipfile import *
    z=ZipFile('sample.zip','r')
    z.extractall('D:\\Academics\\Python')
```

Random Accessing of Binary files using mmap

The full form of mmap is 'memory mapped file'. Once a binary file is created with some data, that data is viewed as strings and can be manipulated using mmap module. The mmap can be used as:

```
    mm=mmap.mmap(f.fileno(),0)
```

This will map the currently opened file 'f' with the file object 'mm'. The 'f' represents the actual binary file that is being mapped. The second argument is zero (0), it means the whole file should be considered for mapping.

Example: #Program to create phone book

```
    with open('phonebook.dat','wb')as f:
        n=int(input('How many records to enter?: '))
        for i in range(n):
            name=input('Enter Name: ')
            phone=input('Enter Number: ')
            name=name.encode()   #Converting name from string to bytes
            phone=phone.encode() #Converting name from string to bytes
            f.write(name+phone)  #Writing data to the file
```

#Performing various operations on a binary file

```
    import mmap, sys
    #display a menu to the users
    print('1 To view all the entries')
    print('2 To view phone numbers')
    print('3 To modify an entry')
    print('4 To exit')
    ch=input('Enter your choice: ')
    if ch=='4':
```

```
        sys.exit()
with open('phonebook.dat','r+b') as f:
        mm=mmap.mmap(f.fileno(),0) #mempry map the the file, 0 means whole file

        if(ch=='1'):   #Display the entire file
                print(mm.read().decode())

        if(ch=='2'):   #Display phone number
                name=input('Enter name: ')
                n=mm.find(name.encode())   #returns the position of name in the file
                n1=n+len(name)          #Go to end of name
                ph=mm[n1:n1+10]         #Display the next 10 bytes
                print('Phone no: ', ph.decode())

        if(ch=='3'):   #Modify phone number
                name=input('Enter name: ')
                n=mm.find(name.encode())   #returns the position of name in the file
                n1=n+len(name)          #Go to end of name
                ph1=input('Enter new phone number: ')   #Taking new phone number
                mm[n1:n1+10]=ph1.encode()  #The old phone no. is 10 bytes after n1
        mm.close()
```

Regular Expression

Character	Description
\w	Any alphanumeric (A-Z, a-z, 0-9)
\W	Any non-alphanumeric
\d	Any digit 0-9
\D	Any non-digit
\s	White space
\S	Non-white space
\b	Space around words
\A	Matches at the start of the string
\Z	Matches at the end of the string

```
#Regular expression (also known as re)

#str = r'm\w\w'
#r is for regular expression , rowstring
#m = first character must start with m
#\w = it means any alpha numeric character followed by m
# str1 = 'This will be printed on the \n new line'
# print(str1)
# str1 = r'This will be printed on the \n new line'
# print(str1)
```

#Using Compile method of re module to compile the regular expresssion

```
# import re
# prog = re.compile(r'm\w\w')
# str1 = 'net cat mat matter' #Regiar expression will act on this
# result = prog.search(str1) #Use of search(), to search the expression in the string
# print(result.group())    #Use of group() to display the result
# str2 = 'later or sooner truth of man matters'
# result = prog.search(str1) #Reusing the regular expression earier
# print(result.group()) #Use of group() to display the result
```

#Create a regular expression to search for string with 'm' a having total 3 character using findall()

```
# import re
# #str1 = input('Enter string:')
# str1 = 'net cat mat matter'
# result = re.findall(r'm\w\w\w',str1)
# print(result)
# #findall() returns always a list
```

#Use of split()

#the spilt() splits the given string into pieces according to the regular expression and returns the pieces as elements of list.

```
#re.spilt(r'\W+',str1)
#Here, the 'Capital W' represents any non-alpha numeric.
# import re
# str1 = 'Hello! All, Good; morning'
# result = re.split(r'\W+',str1)
# print(result)
```

#Create a regular expression that represents a string with the new string

```
# import re
# str1 = 'AITS is the best Organization'
# print(str1)
# result = re.sub(r'AITS','Atmiya University',str1) #Use of sub()
# print(result)
```

#Create a regular expression that finds the words strating with 's'

```
# import re
# str1 = 'Sun shines sooner or later'
# result = re.findall(r's[\w]*',str1)
```

```
# #result = re.findall(r's\w+',str1)
# print(result)
```

#Create a regular expression that finds the words starting with a number

```
# import re
# str1 = 'The special classess are arranged on 11th and 21st of every month'
# result = re.findall(r'\d[\w]*',str1)
# print(result)
```

#Create a regular expression that retrive the words having 5 characters.

```
# import re
# str1 = 'sun mon tues wedn thurs frida saturday'
# result = re.findall(r'\b\w{5}\b',str1)
# print(result)
```

#Create a regular expression that retrive the words having 4 or 5 characters.

```
# import re
# str1 = 'sun mon tues wedn thurs frida saturday'
# result = re.findall(r'\b\w{4,5}\b',str1)
# print(result)
```

#Create a regular expression that retrive the words having 4 or more than 4 and less than 7 characters.

```
# import re
# str1 = 'sun mon tues wedn thurs friday saturday'
# result = re.findall(r'\b\w{4,6}\b',str1)
# print(result)
```

#create aregular expression that retrive words having 2 or more than 2 and less than 6 cgharacters

```
# import re
# str1 = 'su mon tues wedn thurs friday saturday'
# result = re.findall(r'\b\w{2,6}\b',str1)
# print(result)
```

#Create a regular expression that retrives only single digit number from the string

```
# import re
# str1 = 'seven eight 9 10'
# result = re.findall(r'\b\d\b',str1)
```

```
# print(result)
```

#Create a regular expression which retrive last wor of the string if it strats with 's'

```
# import re
# str1 = 'seven eight nine seventeen'
# result = re.findall(r's[\w]*\Z',str1)
# print(result)
```

#Create a regular expression which retrive first word of the string if it strats with 's'

```
# import re
# str1 = 'seven seen eight nine seventeen'
# result = re.findall(r'\As[\w]*',str1)
# print(result)
```

#Create a regular expression that retrive the enrollment number of students

```
# import re
# str1 = 'abc 1234'
# result = re.findall(r'\d+',str1)
# print(result)
```

#Create a regular expression that retrive the name of students

```
# import re
# str1 = 'abc 1234'
# result = re.findall(r'\D+',str1)
# print(result)
```

#Create a regular expression that retrive the word strting with 'st'

```
# import re
# str1 = 'summer sun strong stunning strom small smell'
# result = re.findall(r'st[\w]*',str1)
# print(result)
```

#Create a regular expression that retrive the word strting with 'st' and 'sm'

```
# import re
# str1 = 'summer sun strong stunning strom small smell'
# result = re.findall(r's[t,m][\w]*',str1)
# print(result)
```